

COMP 3522

OBJECT ORIENTED PROGRAMMING 2

Midterm Exam

February 2025

Full Name	
Signature	
Student ID	
Set	

Instructions:

1. This is a closed-book exam. One cheat sheet allowed.
2. Headphones are OK.
3. This midterm is 90 minutes long.
4. No candidate will be admitted after 30 minutes.
5. No candidate will be permitted to leave during the first 30 minutes.
6. Add The answers for T/F and MCQ questions to the Answer sheet.
7. Good luck. You're awesome, you've got this.

Final grade:

MCQ Questions

[8 Marks]

Question 1

Which of the following statements about Python is FALSE?

- A) Python supports multiple programming paradigms, including OOP, functional, and procedural programming.
- B) Python is a compiled language, ensuring optimized performance.
- C) Python uses dynamic typing, meaning variable types are determined at runtime.
- D) Python has a comprehensive standard library with built-in functions for various operations.

Question 2

What is the primary difference between abstraction and encapsulation?

- A) Abstraction is about bundling data together, while encapsulation is about hiding data.
- B) Encapsulation is about bundling data together, while abstraction is about hiding details.
- C) Abstraction is used only in procedural programming, while encapsulation is exclusive to OOP.
- D) Encapsulation only applies to methods, whereas abstraction applies only to data.

Question 3

What happens if a class does not implement an abstract method from an abstract base class (ABC)?

- A) Python automatically implements default behavior.
- B) The program will compile but raise an error at runtime.
- C) An error occurs when trying to instantiate the subclass.
- D) The program runs normally.

Question 4

What is the primary benefit of decoupling a UI from its data source using an interface?

- A) It increases the number of dependencies in the system.
- B) It ensures the UI can work with multiple types of data sources without modification.
- C) It makes the UI run faster.
- D) It eliminates the need for an underlying data source.

Question 5

What is the primary indication that a class violates the Single Responsibility Principle (SRP)?

- A) The class is too short and does not perform enough tasks.
- B) The class has multiple reasons to change.
- C) The class contains too many private variables.
- D) The class only interacts with one other class.

Question 6

What is the output of the following function call?

```
def my_function(*args):  
    print(args)  
  
my_function(1, 2, 3)
```

- A) (1, 2, 3)
- B) [1, 2, 3]
- C) {1, 2, 3}
- D) 1, 2, 3

Question 7

Given the following class hierarchy, what is the correct MRO for class Z?**

```
class X: pass
class M(X): pass
class N(X): pass
class P(X): pass
class Q(X): pass
class R(X): pass
class S(X): pass
class K1(M, N, P): pass
class K2(Q, N, R): pass
class K3(Q, M, S): pass
class Z(K1, K2, K3): pass
```

- A) [Z, K1, K2, K3, M, N, P, Q, R, S, X]
- B) [Z, K1, K2, K3, Q, M, N, P, R, S, X]
- C) [Z, K1, M, N, P, K2, Q, R, K3, S, X]
- D) TypeError: Cannot determine MRO

Question 8

What happens if two Enum members have the same value?

```
from enum import Enum
class Colors(Enum):
    RED = 1
    BLUE = 2
    GREEN = 1
```

- A) A TypeError occurs.
- B) The second assignment (GREEN = 1) overwrites RED = 1.
- C) The Enum assigns the same name to both members.
- D) The Enum keeps only the first occurrence (RED = 1), and GREEN becomes an alias for RED.

Question 9

How is a generator expression different from a list comprehension?

- A) A generator expression always returns a list, whereas a list comprehension returns an iterator.
- B) A generator expression returns an iterator, whereas a list comprehension returns a list.
- C) Generator expressions are faster than list comprehensions in all cases.
- D) A generator expression is more readable than a list comprehension.

Question 10

Which of the following best describes the purpose of the `cumttime` (cumulative time) column in a profiling report?

- A) It measures the total time spent in the function, excluding sub-functions.
- B) It calculates the number of function calls per second.
- C) It measures the total time spent in the function, including all sub-function calls.
- D) It estimates the memory footprint of function execution.

True/False Questions

[4 Marks]

Question 1

True or False?

Dictionaries in Python can have immutable objects as keys, such as tuples.

Question 2

True or False?

The `timedelta` class in the `datetime` module is used to perform arithmetic operations on datetime objects.

Question 3

True or False?

The `repr` Function is meant for display purposes and should be used when you want a string that is informative and readable to humans whereas `str` function is meant for debugging and development.

Question 4

True or False?

The Interface Segregation Principle (ISP) advocates that classes should implement large, general-purpose interfaces to minimize the number of interfaces in a system.

Question 5

True or False?

The Dependency Inversion Principle (DIP) recommends that high-level modules should depend on low-level modules, ensuring that the high-level modules can directly control the low-level functionality.

Question 6

True or False?

When using multiple inheritance in Python, a function call that relies on `super()` may invoke methods from classes that are not explicitly listed in the class definition.

Question 7

True or False?

It is recommended to use `multiple if` checks instead of exceptions because exception handling is slower in Python.

Question 8

True or False?

An object must implement both `iter()` and `next()` to be considered an iterable in Python.

Question 9

True or False?

The `map()` function applies a given function to each element of an iterable and returns a list of results.

Question 10

True or False?

Statistical profiling is the method used by Python's built-in `cProfile` module.

What is the output

[2 Marks]

What is a possible output for the following code?

```
from abc import ABC, abstractmethod
import random

# ----- SUBJECT (Observable) -----
class StockExchange:
    """StockExchange acts as the Subject, notifying observers about stock price changes."""

    def __init__(self):
        self._observers = [] # List of registered observers
        self._stocks = {} # Stock prices

    def register_observer(self, observer):
        """Registers a new observer."""
        if observer not in self._observers:
            self._observers.append(observer)

    def remove_observer(self, observer):
        """Removes an observer."""
        if observer in self._observers:
            self._observers.remove(observer)

    def notify_observers(self, stock_symbol, new_price):
        """Notifies all observers about a stock price change."""
        for observer in self._observers:
            observer.update(stock_symbol, new_price)

    def set_stock_price(self, stock_symbol, new_price):
        """Updates stock price and notifies observers."""
        old_price = self._stocks.get(stock_symbol, None)
        self._stocks[stock_symbol] = new_price

        # Notify observers only if the price has changed
        if old_price is None or old_price != new_price:
            print(f"\n Stock Update: {stock_symbol} new price is ${new_price:.2f}")
            self.notify_observers(stock_symbol, new_price)

# ----- OBSERVERS -----
class Observer(ABC):
    """Abstract Observer class"""

    @abstractmethod
    def update(self, stock_symbol, new_price):
        """Receive stock update"""
        pass

class InvestorObserver(Observer):
    """Observer that represents an investor tracking a single stock."""

    def __init__(self, name, stock_symbol):
        self.name = name
        self.stock_symbol = stock_symbol

    def update(self, stock_symbol, new_price):
        """React to stock price update"""
        if stock_symbol == self.stock_symbol:
            print(
                f"Investor {self.name}: {stock_symbol} changed to ${new_price:.2f}"
            )

class NewsAgencyObserver(Observer):
    """Observer that reports significant stock changes to the media."""

    def update(self, stock_symbol, new_price):
        """Broadcast news when price fluctuates significantly"""
        if random.choice([True, False]): # Simulate selective reporting
```

```

        print(
            f"News Flash: {stock_symbol} hit ${new_price:.2f}! Market impact expected."
        )

class PortfolioObserver(Observer):
    """Observer that monitors a portfolio of stocks for an investor."""

    def __init__(self, investor_name, stocks):
        self.investor_name = investor_name
        self.stocks = stocks # Dictionary of stock symbols and price thresholds

    def update(self, stock_symbol, new_price):
        """Check if stock price in portfolio needs attention"""
        if stock_symbol in self.stocks:
            threshold = self.stocks[stock_symbol]
            if new_price >= threshold:
                print(
                    f"{self.investor_name}'s Portfolio Alert: {stock_symbol} has reached ${new_price:.2f}"
                )

# ----- DEMO RUN -----
if __name__ == "__main__":
    stock_exchange = StockExchange()

    # Create observers
    investor1 = InvestorObserver("Alice", "AAPL")
    investor2 = InvestorObserver("Bob", "TSLA")
    news_agency = NewsAgencyObserver()
    portfolio = PortfolioObserver("Charlie", {"AAPL": 150, "TSLA": 700, "GOOGL": 2800})

    # Register observers
    stock_exchange.register_observer(investor1)
    stock_exchange.register_observer(investor2)
    stock_exchange.register_observer(news_agency)
    stock_exchange.register_observer(portfolio)

    # Simulate stock price updates

    # AAPL price increase
    stock_exchange.set_stock_price("AAPL", 150)

```

Your Answer:

```

Stock Update: AAPL new price is $150.00
Investor Alice: AAPL changed to $150.00
News Flash: AAPL hit $150.00! Market impact expected.
Charlie's Portfolio Alert: AAPL has reached $150.00

```

or

```

Stock Update: AAPL new price is $150.00
Investor Alice: AAPL changed to $150.00
Charlie's Portfolio Alert: AAPL has reached $150.00

```

Refactoring Question

[4 Marks]

In *Code Snippet A*, You are given a class `ShoppingCartWithoutSOLID` that calculates the final price of a purchase based on a given discount type. The class currently uses conditional statements (if-elif) to handle different discount types.

Your task is to refactor the given code by applying the SOLID principles to improve maintainability, scalability, and flexibility. Your refactored design should allow easy addition of new discount types without modifying the core `ShoppingCart` class.

Code Snippet A:

```
class ShoppingCartWithoutSOLID:
    def checkout(self, total_amount, discount_type, discount_value=0):
        if discount_type == 'none':
            final_amount = total_amount
        elif discount_type == 'percentage':
            final_amount = total_amount - (total_amount * discount_value / 100)
        elif discount_type == 'fixed':
            final_amount = max(0, total_amount - discount_value)
        else:
            raise ValueError("Unknown discount type")
        print(f"Total after discount: ${final_amount:.2f}")

cart_no_solid = ShoppingCartWithoutSOLID()
cart_no_solid.checkout(100, 'none')
cart_no_solid.checkout(100, 'percentage', 10)
cart_no_solid.checkout(100, 'fixed', 15)
```

To improve the design, refactor the code by following SOLID principles. Your solution should:

- Define an **abstract class or interface** that will serve as a blueprint for different discount types.
- Implement three separate classes, each handling a specific discount type:
 - NoDiscount
 - PercentageDiscount
 - FixedAmountDiscount
- Modify the `ShoppingCartWithoutSOLID` class into a **new class `ShoppingCart`** that follows the **Dependency Inversion Principle (DIP)** by depending on an abstraction instead of concrete discount logic.

Question 1

Use the provided code snippets to finish the refactoring process. You may use arrows or label the snippets with numbers or letters to indicate the order of execution.

Answer:

```
from abc import ABC, abstractmethod

# Strategy Interface

class DiscountStrategy(ABC):
    @abstractmethod
    def apply_discount(self, amount):
        pass

# Concrete Strategies

class NoDiscount(DiscountStrategy):
    def apply_discount(self, amount):
        return amount

class PercentageDiscount(DiscountStrategy):
    def __init__(self, discount_value):
        self.discount_value = discount_value

    def apply_discount(self, amount):
        return amount - (amount * self.discount_value / 100)

class FixedAmountDiscount(DiscountStrategy):
    def __init__(self, discount_value):
        self.discount_value = discount_value

    def apply_discount(self, amount):
        return max(0, amount - self.discount_value)

# Context

class ShoppingCart:
    def __init__(self, discount_strategy: DiscountStrategy):
        self.discount_strategy = discount_strategy

    def checkout(self, total_amount):
        final_amount = self.discount_strategy.apply_discount(total_amount)
        print(f"Total after discount: ${final_amount:.2f}")
```

```
self.discount_strategy = discount_strategy
```

```
@abstractmethod
def apply_discount(self, amount):
    pass
```

```
class PercentageDiscount(DiscountStrategy):
```

```
class FixedAmountDiscount(DiscountStrategy):
```

```
        final_amount =
self.discount_strategy.apply_discount(total_amount)
```

```
class DiscountStrategy(ABC):
```

```
def checkout(self, total_amount):
```

```
class NoDiscount(DiscountStrategy):
```

```
        print(f"Total after discount:
${final_amount:.2f}")
```

```
def apply_discount(self, amount):
    return amount
```

```
def __init__(self, discount_value):
    self.discount_value = discount_value
```

```
def apply_discount(self, amount):
    return amount - (amount * self.discount_value / 100)
```

```
def apply_discount(self, amount):
    return max(0, amount - self.discount_value)
```

```
def __init__(self, discount_strategy: DiscountStrategy):
```

```
class ShoppingCart:
```

Question 2

Demonstrate how to use the new ShoppingCart class to apply different types of discounts. The old usage, before refactoring, is shown on the right column.

Your Answer:

```
# Usage
cart = ShoppingCart(NoDiscount())
cart.checkout(100)

cart = ShoppingCart(PercentageDiscount(10))
cart.checkout(100)

cart = ShoppingCart(FixedAmountDiscount(15))
cart.checkout(100)
```

Before Refactoring

```
# Usage
cart_no_solid = ShoppingCartWithoutSOLID()
cart_no_solid.checkout(100, 'none')
cart_no_solid.checkout(100, 'percentage', 10)
cart_no_solid.checkout(100, 'fixed', 15)
```

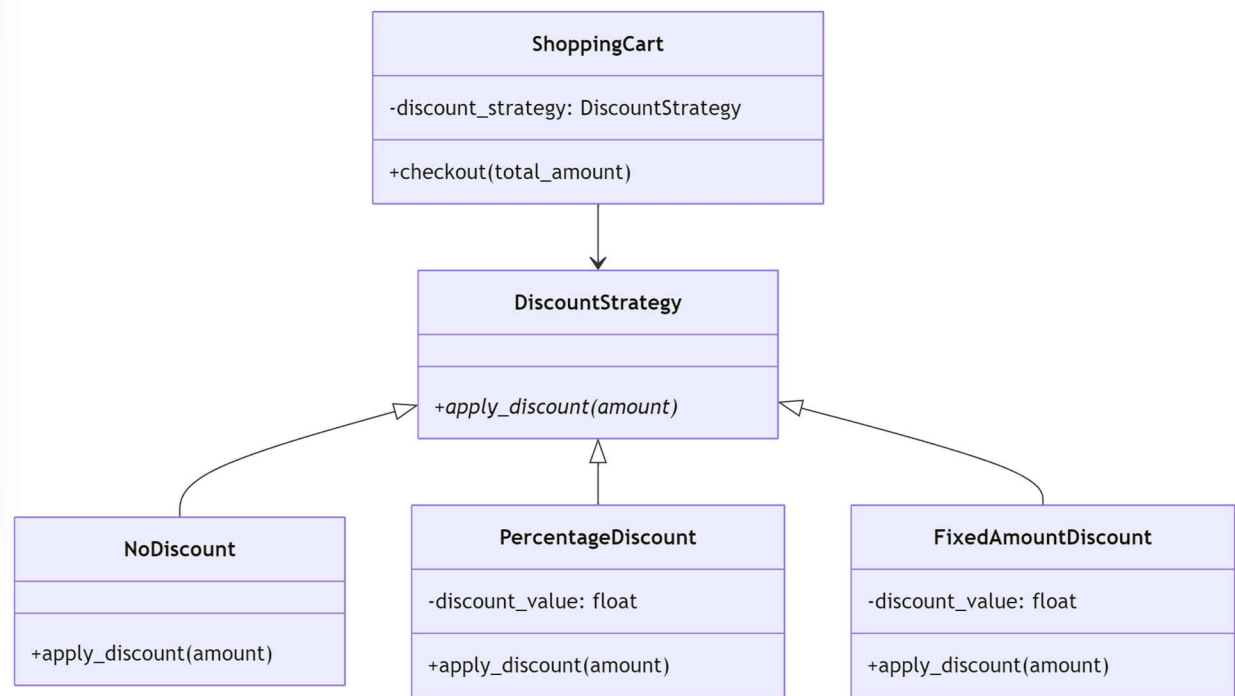
Question 3

Discuss how your refactored design adheres to the Single Responsibility Principle (SRP), Open-Closed Principle (OCP), and Dependency Injection Principles (DIP). Explain how it improves code maintainability compared to the original implementation.

Answer: by using the abstract class `DiscountStrategy` and concrete implementations `NoDiscount`, `PercentageDiscount`, and `FixedAmountDiscount`, we have separated the discount logic into distinct classes. This adheres to the SRP by ensuring that each class has a single responsibility related to discount calculation. The `ShoppingCart` class now depends on an abstraction (`DiscountStrategy`) rather than concrete discount logic, adhering to the DIP. This design follows the OCP as new discount types can be added by creating new classes that implement the `DiscountStrategy` interface without modifying the existing code. This approach improves maintainability by allowing easy extension and modification of discount types without affecting the core `ShoppingCart` class.

Question 4

Complete the following UML class diagram showing the relationships between the `ShoppingCart` class and the discount classes.



THE END 😊